Control Panel

Select Model Engine

gemini-2.5-pro

System: Vertex AI (ADC Enabled)

IBM Legacy Modernization Agent v2.0

Legacy Scribe

1a. Upgrading large-scale legacy systems into intelligent, scalable, future-ready platforms.

Legacy Asset Input

Asset Type

Legacy SQL Schema

Paste Code / Logs here:

```
[SCRIBE]: Deconstructing legacy syntax and state machine...
```

Modernization Strategy Generated

System Analysis

Logic Summary: The legacy DDL script defines a simple database table, CUST_REC, to store basic customer information including a 10-character ID, a name, and a credit limit. This table is the foundational entity for customer data storage.

Identified Security Vulnerabilities

```
CREATE TABLE CUST_REC (
 ID CHAR(10),
 NM VARCHAR(50),
 CRD_LMT DECIMAL(10,2)
 );
```

Initiate Modernization Pipeline

• Lack of a defined Primary Key on the ID column, which allows for duplicate customer records and severe data integrity issues.

• Absence of indexes, which will lead to poor query performance and full table scans on large datasets, making the system vulnerable to denial-of-service through resource exhaustion.

• Use of fixed-length CHAR for the ID, which is inefficient for storage if IDs vary in length and can be a minor information disclosure vector (e.g., padding).

• No NOT NULL constraints on critical fields like ID and NM, permitting incomplete or invalid records which can crash downstream applications.

Target Architecture: This data entity should be managed by a dedicated 'Customer' microservice. This service would expose a RESTful API (e.g., using FastAPI) for all CRUD (Create, Read, Update, Delete) operations. The underlying database schema should be modernized by using a UUID or a sequence-generated integer as the primary key, adding appropriate indexes (e.g., on the name field), enforcing NOT NULL constraints, and including

audit columns like 'created_at' and 'updated_at' for traceability.

Refactored Implementation (Python/FastAPI)

```
from fastapi import FastAPI, HTTPExcep
from pydantic import BaseModel, Field
from datetime import datetime
from decimal import Decimal
import uuid

app = FastAPI(
    title="Customer API",
    description="API for managing cust
    version="1.0.0"
)

# Modern Pydantic model for data valic
class Customer(BaseModel):
    id: uuid.UUID = Field(default_fact
    name: str = Field(..., max_length=
    credit_limit: Decimal = Field(...,
    created_at: datetime = Field(defau
    updated_at: datetime = Field(defau
```

```
    class Config:
        orm_mode = True

# In-memory dictionary to act as a sim
db: dict[uuid.UUID, Customer] = {}

@app.post("/customers/", response_mode
async def create_customer(customer: Cu
    """
    Creates a new customer record.
    In a real application, this would
    """
    if customer.id in db:
        raise HTTPException(status_coc
    db[customer.id] = customer
    return customer

@app.get("/customers/{customer_id}", r
async def get_customer(customer_id: uu
    """
    Retrieves a single customer by the
    """
    if customer_id not in db:
        raise HTTPException(status_coc
    return db[customer_id]
```

Download Modernization Report